lambdaSearch

Project report

INF221 Avansert funksjonell programmering

Martin Elde Knutsen

01. 05 2026

1 Project description

1.1 What was the initial idea / background of the project?

The reason I wanted to make this project is that I really like CLI tools. I use them a lot every day. But I think some of them have unnecessary, bad, and unintuitive syntax. That's why I wanted to make my own tool where I had complete control over the syntax. I also wanted to build a TUI around it, mostly because brick sounded like a fun library to try out. So the idea was to use FFI, MegaParsec, and brick to make a CLI tool for searching for files on my PC. I Also wanted to try to use STM an concurrency to search through the files faster.

1.2 Project goals

- Make the searching program fast, efficient and safe.
- Call C functions to retrieve file metadata, with as little as possible overhead.
- Use the Either monad for error handling.
- Use the STM monad for concurrency.
- Parse CLI input with MegaParsec.
- Parse a configuration file.
- Write good readable code

1.3 Result

- I managed to make the CLI with the syntax that I wanted. But I did not implement concurrency, because it was already faster than the built-in find command and I wanted to prioritise testing and the TUI.

1.3.1 Implemented

- Core directory traversal (filter by regex, extension, hidden, ignored).
- Command execution (-x flag substituting {}).
- TUI with `brick` (live search, vim-bindings, open in `$EDITOR`).
- C FFI bindings (`readdir`, etc.) for fast metadata retrieval.
- CLI parsing using `megaparsec`.
- Error handling with `Either`/`ExceptT` for permission issues.
- test suite (Tasty, QuickCheck, HUnit).

1.3.2 Not Implemented

- Concurrency (STM / parallel traversal).
- Configuration file parsing.

1.4 Future extensions

- Implement concurrent searching

2 Description of the functional programming techniques

2.1 Monadic Parsing Combinators

When parsing for the CLI I use a lot of combinators. Making it very readable and elegant.

- Example:

```
pFlags :: Parser DataFlags
pFlags = choice
    [ SearchPatternFlag <$> ((string' "-p" <|> string  "--pattern"   ) *> sc *> parseWord)
     , HiddenFilesFlag  <$  ( string' "-a" <|> string  "--show--dots")
       ...
```

- We use the fmap operator (<$>) to lift the parsed result into the DataFlags context.
- Next is the actual parser. Three things to notice here:

1. The string' function enables case-insensitive parsing (e.g., both -p and -P).
2. I use the <|> operator, which here sort of acts like a or. It tries to parse -p, and if that fails, it tries to parse –pattern. Because the type signature of this operator is f a -> f a -> f a, you can treat the combined expression as a single parser. This means anywhere you parse one thing, you can easily try to parse alternatives without any hassle. Of course, the inner type a must be the same for both parsers.
3. The sequence operator (*>) consumes and discards the flag prefix. We chain this with sc to skip whitespace before capturing the actual argument.

- For HiddenFilesFlag, we use <$. Since this flag takes no arguments, <$ directly replaces the parsed string with the data constructor.
- And it's all wrapped in choice which acts like a sequence of <|>

2.2 Monad Transformers and Monadic Error Handling

To make the directory traversal as fast as possible, I used the Foreign Function Interface (FFI) to bind directly to C POSIX functions. But to keep the Haskell side safe, I wrapped these impure calls in Monad Transformers to handle error and end of dir exceptions.

- Example:

```
type DirContentT = ExceptT DirError IO (Maybe DirContent)

readDirEnt :: DirStream ->  DirContentT
readDirEnt dir = ExceptT readContent
    where
    readContent :: IO (Either DirError (Maybe DirContent))
    readContent = do
      -- ... raw IO and FFI calls to c_readdir ...
```

- First, I define DirContentT using ExceptT. This layers an exception context (DirError) over the IO monad. This lets me sequence IO actions but still cleanly short-circuit if a specific directory error happens (like a permission denied error).
- Inside readContent, I do the actual raw IO calls to the C function c_readdir. Depending on the Errno returned by C, I can return pure . Right $ Nothing (if we hit the end of the folder) or pure . Left $ ReadDirErr err if it actually failed.

2.3 Higher-Order Functions and Folds

2.3.1 In the parser

- To keep track of which flags the user passed in the CLI, I used a functional fold. This is a clean way to build up the configuration state purely.
- Example:

```haskell
parseFlags :: Parser Arguments
parseFlags = do
  fs <- parseAllFlags
  pure $ foldl' applyFlag emptyFilterFlags fs
```

- Essentially what a fold is is a way consume a recusiv datastructure by replacing the constructor with a function. In the example above we the replace : in [DataFlags] with applyFlag.
- Why strict fold? Because we want to avoid space leaks with accumulating expressions on the heap. It is probably not an issue on my program, but it is just good practice.
- The magic happens with the applyFlag function, which has the type signature Arguments -> DataFlags -> Arguments. It basically acts as a state transition function. It takes the current Arguments state, looks at the new DataFlags we just parsed, and returns a newly updated Arguments record.
- So we start with emptyFilterFlags as our base state, consume the list of parsed flags, and get our fully constructed configuration. Quite a nite solution i think

2.3.2 In the traversal function

First, Haskell is perfect for these traversal functions because it makes so much sense to write them recursively. You simply apply the same logic to every directory, accumulating the results as you search.

In the main function, where I do the traversals, I use higher-order functions and a generalized fold function.

```haskell
foldDirectoryTree
    :: (a -> RawFilePath -> DirContent -> IO a)
    -> a --
    -> RawFilePath
    -> IO a
foldDirectoryTree foldFunc acc rootPath  = do
    isDir <- isDirectory <$> getFileStatus  rootPath
    if not isDir
        then pure acc
        --  look at code for traverseDirectoryContents implementation
        else traverseDirectoryContents innerloop acc rootPath
    where
        innerloop currentAcc dc@(typ,filename) = do
            let filePath = rootPath  </> filename
            let isDir = typ == dtDir
            nextAcc <- foldFunc currentAcc rootPath dc

            if not isDir
                then pure nextAcc
                else foldDirectoryTree foldFunc nextAcc filePath
```

- foldDirectoryTree extends the folding to a rose three. Here it defines canonical way to consume the structure. In this case the stucture is our filesystem and we have a function that telles how to accumulate the values.
- The cool thing is that it guarantees correct threading of the accumulator. a is just a polymorphic type variable the compiler has no information about what it is. Because of this, the only way foldDirectoryTree can ever produce a new a for the next recursive call is by calling foldFunc. It literally cannot do anything else with it not inspect it, not drop it, not make one from thin air. The compiler simply does not have enough information to express any of those operations. This property is called Parametric Polymorphism, and it gives us a guarantee that the fold does no funky business with our state.
- I would argue this is much safer than typing a as something concrete like [RawFilePath]. The moment you do that, you suddenly open up a whole world of possible manipulation the function could reorder the list, drop entries etc..
- However. Sadly we are still inside the IO monad though, so the guarantee is not airtight. But we are still sure that it does folds correctly and that is does not do anything bad with a. And as long as we write the foldFunc we are good.

I also want to mention that having this generic signature is very useful, we can reuse the same traversal logic for completely different purposes just by swapping a:

```
traverseRecursively :: Arguments -> [FileInformation] -> RawFilePath -> IO
[FileInformation]
traverseRecursively args = foldDirectoryTree foldFunc
    where
    regexCompiled = compileRegexFilter args
    foldFunc :: [FileInformation] -> RawFilePath -> DirContent -> IO [FileInformation]
    foldFunc acc parentPath dc@(typ,file)  = ...
```

Or i can use it to count the files

```
countFiles :: Integer -> RawFilePath -> IO Integer
countFiles  = foldDirectoryTree foldFunc
    where
    foldFunc ::  Integer -> RawFilePath -> DirContent -> IO Integer
    foldFunc s  _ ((== dtDir) -> True ,_) = pure (1 +  s)
    foldFunc s _ _                        = pure s
```

2.4 Algebraic data types

For storing the config of my search logic i used records.

- Example:

```
data Arguments = Arguments {
      regxPattern    :: Maybe SearchPattern
    , exclude        :: Maybe SearchFilters
    , extension      :: Maybe Extention
    , hideHidden     :: Bool
    , appliedCommand :: Maybe ConstructedCommand
}  deriving (Eq, Show)
```

- The reason why Haskell datatypes are algebraic is that they are built from two operators sums and products. In this case, arguments form a product type, and Maybe from a sum type defined as data Maybe a = Nothing | Just a, which can hold $1 + |a|$ values. By using Maybe instead of f.eks. "", we encode optionality into the type. And now, it is structurally impossible to have an invalid state.

- The compiler then enforces via pattern matching exhaustiveness checking if every call site handles both the present and absent case.

```
data Args = Text String | PathToSubs
    deriving (Eq,Show)

type ConstructedCommand = (String, [Args])
```

- When you give the command to execute, you do it like this: cat {}. So here, it is very beneficial to create a datatype for this that says the argument is either a flag or a path where you substitute in the actual filepath. And a complete command is just a list of these.

2.5 View Patterns

I made use of ViewPatterns language extension to keep my pattern matching concise. It lets me evaluate a function directly inside the pattern match, saving me from writing nested case expressions.

- Example:

```
getExtentionFilter :: FilterFlags -> RawFilePath ->  Bool
getExtentionFilter sf fp = case extension sf of
                        Nothing    ->  True
                        (Just ext) -> case getFileExtention fp of
                                        Nothing     -> False
                                        (Just curr) -> curr == ext
```

- Becomes:

```
getExtentionFilter :: Arguments -> RawFilePath -> Bool
getExtentionFilter (extension -> Nothing) _                                 = True
getExtentionFilter (extension -> Just _ )  (getFileExtention -> Nothing)    = False
getExtentionFilter (extension -> Just ext) (getFileExtention -> Just curr) = curr == ext
```

- I think this reads a lot better than the case statement, because it immediately tells what output you get on that specific input. However this is just my subjective opinion.

2.6 A little note about FFI

Another thing i want to talk about i the calls done with FFI

```
foreign import ccall safe "readdir"
  c_readdir :: Ptr CDir -> IO (Ptr CDirent)

foreign import ccall unsafe "__hscore_d_name"
  c_name :: Ptr CDirent -> IO CString

foreign import ccall unsafe "__posixdir_d_type"
  c_type :: Ptr CDirent -> IO DirType
```

- As you can see, c_readdir uses a safe call, but the other two use unsafe. Why is this? Normally, when we make a safe call with the FFI, the runtime releases the capability before doing any C stuff. This allows any other Haskell thread to grab the capability (basically the «right to run Haskell code»). This, of course, results in a bit of overhead. When we do an unsafe call, it skips all of this. This can result in blocking the entire Haskell execution until C returns. So, if the C function takes a long time, or if it does not return, it can lead to a deadlock. It is also very important to use safe calls when the C code calls back into Haskell (not an issue here). So, it's important that we are selective with the functions we call `unsafe` with, and we should not use them for anything that can take a substantial amount of time (like networked file systems or slow spinning disks). Issue talking about this. This is why, when we do the `readdir`, we do it as a `safe` call. The other two are safe to make `unsafe` because they just read a field of a struct, which is very fast, and they do not do anything I/O related no syscalls, so there is no waiting.

3 Self evaluation:

3.1 What was positive about working on this project?

- I think that using the things we learn in the lessons in practice is really useful. easter egg
- I think it is hard to appreciate how good and useful Haskell's type-system is before you write a bigger project like this. When you write more and more, Haskell's type-system that goes from being something that maybe restricts to something that guides you and holds your hand while you write code.
- It is also very useful to think functionally about problems and appreciate how much shorter and more elegant solutions are when you just solve them with a composition of functions. Now after writing a fair bit of Haskell i found that i am always seeking filter, map, lambda etc. when i write code in java. It makes the code shorter, and in my opinion more readable.
- I have also enjoyed gaining experience with the Haskell ecosystem, specifically Cabal, Hoogle, and Hackage. Hoogle has been particularly useful, the ability to search for functions by their type signatures is brilliant.

3.2 What would you have done differently if you were to do it again?

- I need to put a lot of thought into my record implementation and data types. In this workflow, you define your data structures first and work outward. Starting with a solid foundation here saves an immense amount of work later on.
  - One example of this was when I wanted to print filenames in green text, but only the filename itself, not the full path. Since I had stored everything in a single string, I had to use messy string manipulation to extract the name. It would have been much better if I had stored the filename and the path in separate record fields or even just a tuple from the start.
- Start earlier with the concurrency. I really wanted to implement concurrent searching

4 Instructions

Check the README on gitlab for more info

```
cabal build

# eks.
cabal run lambdaSearch -- . -e hs
cabal run lambdaSearch -- -e hs

# For å kjøre testene
cabal test
```

5 Repo URL

https://git.app.uib.no/martin.e.knutsen/inf221-Semesteroppgave